RETO 1-SISTEMAS DIGITALES 2: CONTRUYE UN MICROPROCESADOR RISC-V

GUÍA 2

En la guía 1 implementó las instrucciones aritmético y lógicas para el modo por direccionamiento por registro (tipo R). En esta guía implementará las instrucciones de la ALU que usan direccionamiento inmediato (tipo I).

PARA ESTUDIO INDEPENDIENTE FUERA DE CLASE

5 – Formatos de instrucción

Según la Tabla de Referencia RISC-V de la página 4 de La Guía Práctica de RISC-V (http://www.riscvbook.com/), RISC-V define 6 tipos de instrucciones que se muestran a continuación:

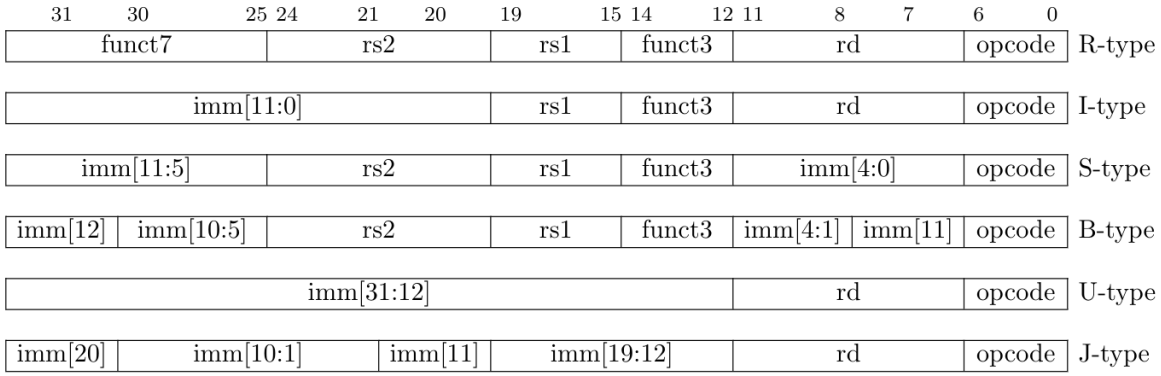


Figura 3 Formatos de instrucción del RISCV y sus respectivos campos

Note que los campos imm corresponden al valor inmediato y aparece no solo en las instrucciones tipo I sino también en todas las instrucciones excepto las tipo R, en formas bastante curiosas.

Como nuestro propósito es soportar todas las instrucciones, es necesario identificar el tipo de instrucción para saber la forma como se deben extraer los campos imm. Para identificar el tipo de instrucción es necesario analizar el campo opcode (bits 6-0 de la instrucción) como se muestra en la siguiente tabla para el RV32I:

instr[4:2] instr[6:5]	000	001	010	011	100	101	110	111
00	I	I	-	-	I	U	I	-
01	S	S	-	R	R	U	R	-
10	R4	R4	R4	R4	R	-	-	-
11	B	I	-	J	I (unused)	-	-	-

Figura 4 Tipos de instrucción según el opcode[6:2]

En la tabla anterior no se consideraron los dos bits menos significativos del opcode (bits 1 y 0), dado a que los diseñadores de RISC-V asumieron que dichos bits siempre deben ser `2'b11` para que sea una instrucción RV32I válida. En nuestro diseño haremos esta verificación activando la señal `is_valid` la cual debe añadir al diseño de la CPU.

PARA EL LABORATORIO

Su misión es, en la sección correspondiente al cableado de la unidad de control (CU: Control Unit), asignar una señal booleana para detectar cada tipo de instrucción. Por ejemplo, para decodificar la instrucción U (resaltada en la Figura 4), puede escribir:

```
assign is_u_instr = (opcode[6:2] == 5'b00101) || (opcode[6:2] == 5'b01101);
```

Luego, termine de escribir el código para determinar los 5 tipos restantes de instrucciones: I, R, S, B, y J:

```
assign is_i_instr = …
assign is_r_instr =
assign is_s_instr =
assign is_b_instr =
assign is_j_instr =
```

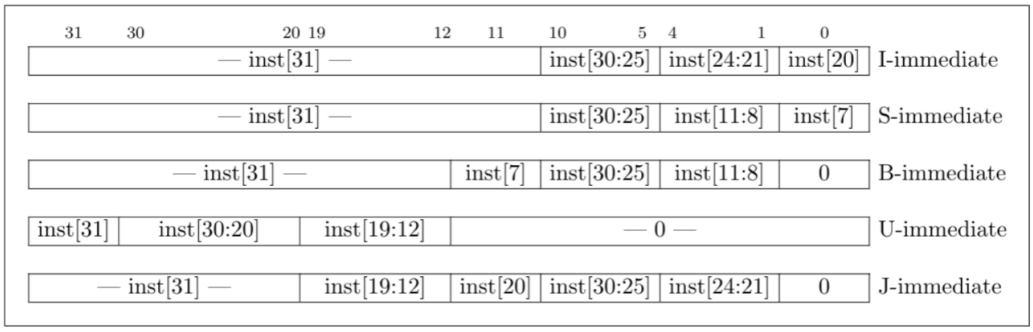
y la señal `is_valid` indicando que la instrucción es válida:

```
assign is_valid =
```

En la guía 1, usted fijó en 1 la señal `wr_en` para habilitar siempre la escritura en el register file. Sin embargo, es hora de cambiarla. Como podrá darse cuenta en la Figura 3, el campo rd, es decir, el registro destino de la operación de la ALU solamente aparece en cierto tipo de instrucciones, por lo tanto, se puede crear la señal `wr_en` a partir de una or que contemple las instrucciones donde el campo rd es válido:

```
assign wr_en = is_r_instr | … ; //Completar
```

Ahora extraeremos el valor inmediato del campo Imm de todas las instrucciones. Como se mostró en la Figura 3, este campo tiene diferentes tamaños y formatos según el tipo de instrucción. Como la ALU opera con números de 32bits y el valor inmediato que se captura de la mayoría de las instrucciones tiene una longitud menor a 32bits, es necesario en todos los casos extender el signo, es decir, el bit más significativo de Imm se replica hasta completar los 32 bits. En la siguiente figura se muestra como reorganizar los bits de Imm para cada tipo de instrucción y el bit con el cual se realiza la extensión de signo.



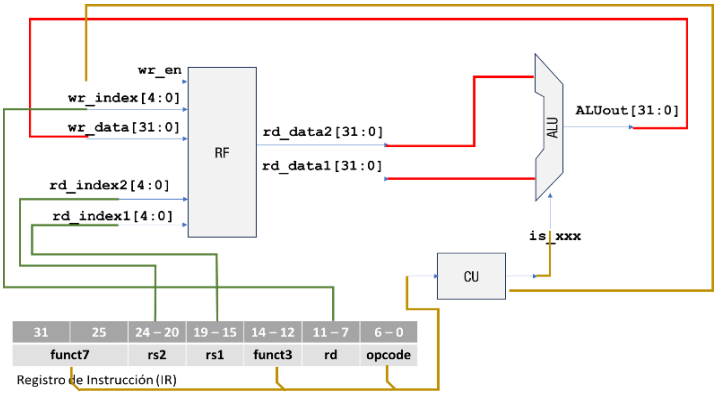
Por ejemplo, para la instrucción tipo I, el bit 31 de la instrucción se debe replicar 21 veces. Una forma de hacer esta implementación es usar un MUX y concatenación de bits de la siguiente manera:

```
assign imm[31:0] = is_i_instr ? {  {21{IR[31]}}, IR[30:20]  } :
            is_s_instr ? {...} :
            ...
            32'b0;  // El valor por defecto de Imm será 0.
```

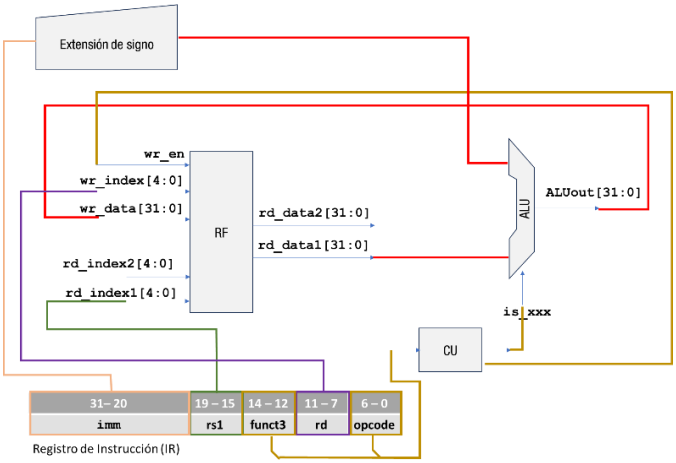
Su misión es completar el MUX para generar la señal `imm` con base en los tipos de instrucción y generar una señal adicional, llamada `imm_valid`, que indique si el campo imm es un campo válido en la instrucción (por ejemplo, si la instrucción es tipo R, este campo no es válido):

```
assign imm_valid = … ;
```

Ahora, es necesario hacer una modificación al cableado entre el Register File y la ALU para soportar las instrucciones tipo I. En la Figura 3 puede notarse que los campos rs1 (registro fuente 1) y rd (registro destino), son válidos para ambos tipos de instrucción, pero el campo rs2 (registro fuente 2) sólo es válido para la instrucción tipo R. Este campo no es válido en la instrucción tipo I, pues en lugar de usar el dato del register file almacenado en la posición rs2, se usa el dato que resulta de la extensión de signo de los bits superiores de la instrucción como se muestra en la siguiente figura de la derecha:



Cableado para implementar las instrucciones tipo R



Cableado para implementar las instrucciones tipo I

Su misión, es incluir un mecanismo para reutilizar la ALU, mezclando el circuito creado en la guía 1 (figura de la izquierda) con el de la derecha, incluyendo solamente señales de control y muy pocos componentes digitales. Ayuda: Modifique la línea de código donde se genera la señal que comanda la entrada b de la ALU y emplee la señal imm_valid.

No olvide modificar la señal is_add, is_and, etc., para habilitar en la ALU la ejecución de las instrucciones addi, andi, etc. Para hacer el cambio, si mira los formatos de instrucción de add y addi en la página 44 de La Guía Práctica de RISC-V (http://www.riscvbook.com/):

31 – 25	24 – 20	19 – 15	14 – 12	11 – 7	6 – 0	
imm[11:0]		rs1	000	rd	0010011	I addi
0000000	rs2	rs1	000	rd	0110011	R add
0100000	rs2	rs1	000	rd	0110011	R sub
funct7			funct3		opcode	

Puede notar que funct3 es idéntico, opcode cambia, y funct7, para la instrucción tipo R, está fusionado con el dato inmediato. Eso significa que el valor de funct7 no importa si la instrucción es tipo I. De esa forma, la señal is_add se debe cambiar por:

```
assign dec_bits = {funct7[5], funct3, opcode};
assign is_add = (dec_bits == 11'b0_000_0110011) ||
                (dec_bits == 11'b0_000_0010011) ||
                (dec_bits == 11'b1_000_0010011);
```

donde la primera condición es para detectar add, y las otras dos fueron para detectar addi, asumiendo que el bit 5 de funct7 puede tomar el valor de 0 o 1 si el tipo de instrucción es inmediata

Recuerde que debe modificar el testbench para probar las instrucciones addi, andi, ori, y xori.